



DevOps

Dr. John-Paul Ore

Housekeeping

- WS-3 Grades Released. Let's thank our TAs Kaushick & Liwen!
- Please remember to turn in your workshop deliverables on time. Workshop deliverables usually include **submitting a form** *and* **having your code on the main branch** of your repo.
- Workshop 6 is due tomorrow (9/30) 11:59pm AOE.

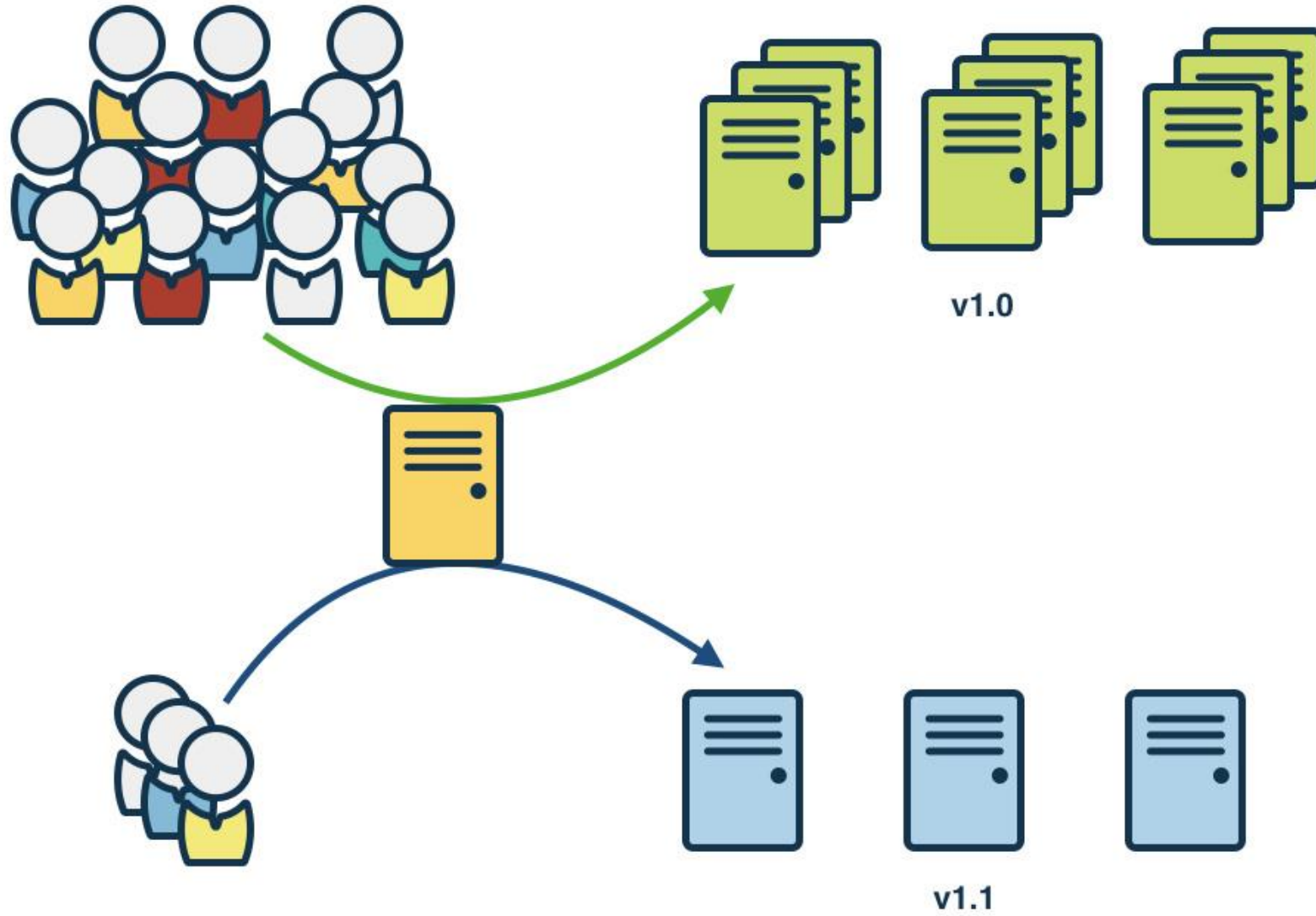
Housekeeping – Regrade Requests

- If necessary, please use the regrade request form at the bottom of the course website to initiate a regrade request: (not by email to the TAs).
- **Workshop days are for workshops:** regrade request conversations happened outside workshop times.
- If you would like additional guidance about why deductions have the point values they do, please direct that conversation to the professor.

Today's Learning Objectives

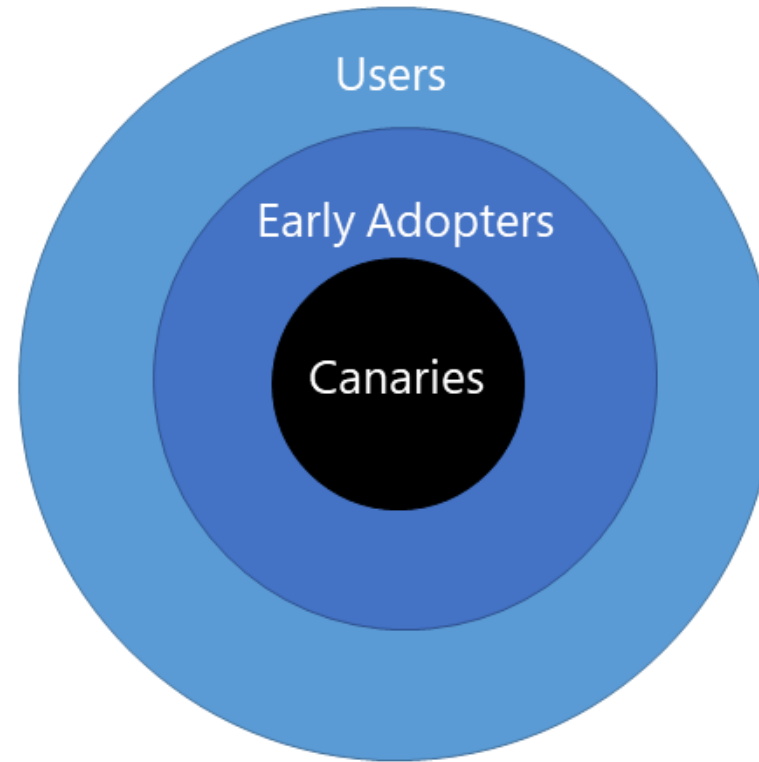
- Deployment Rings / Blast Radius
- Testing Shift Left / Shift Right
- Intro to Infrastructure Concepts
- Review of the 3-Layered Architecture and why it's not enough
- Desirable properties of Modern Web Architecture
- Availability Zones / Regions / Mirroring Sharding
- Circuit Breakers / Content Delivery Networks

Canary Releasing

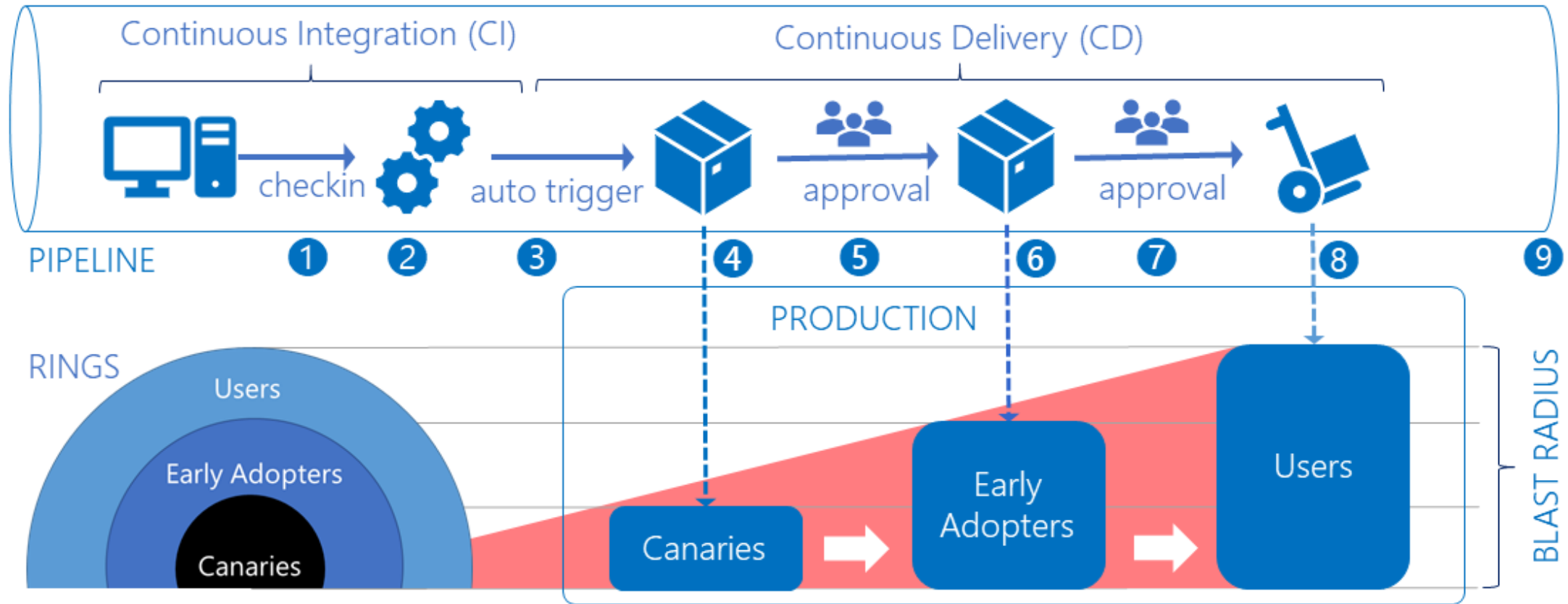


Deployment “Rings” (a.k.a. “Rollout Rings”)

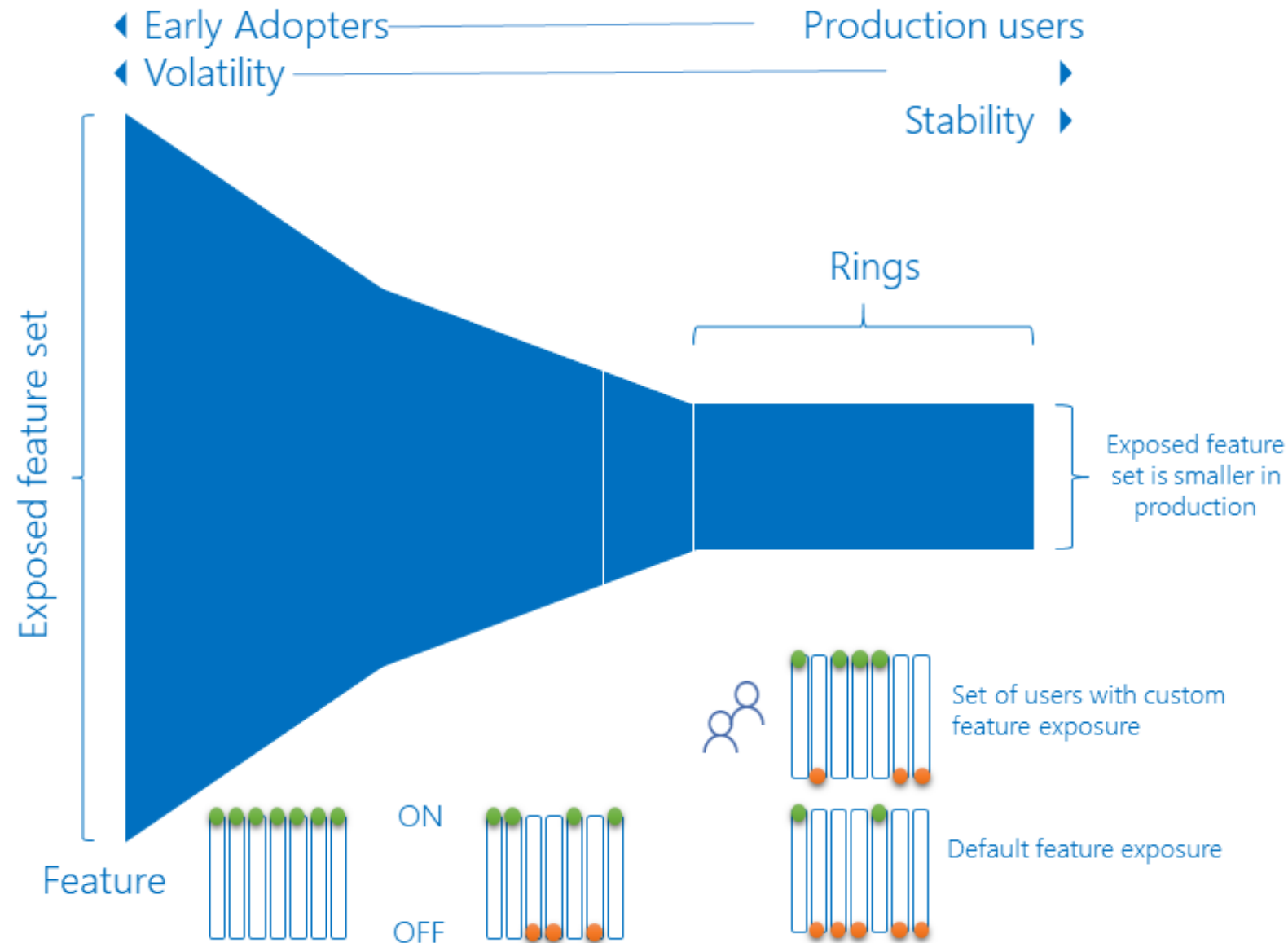
Canaries - first set of users to get access to a new feature



Rollout “Rings” + “Blast Radius”



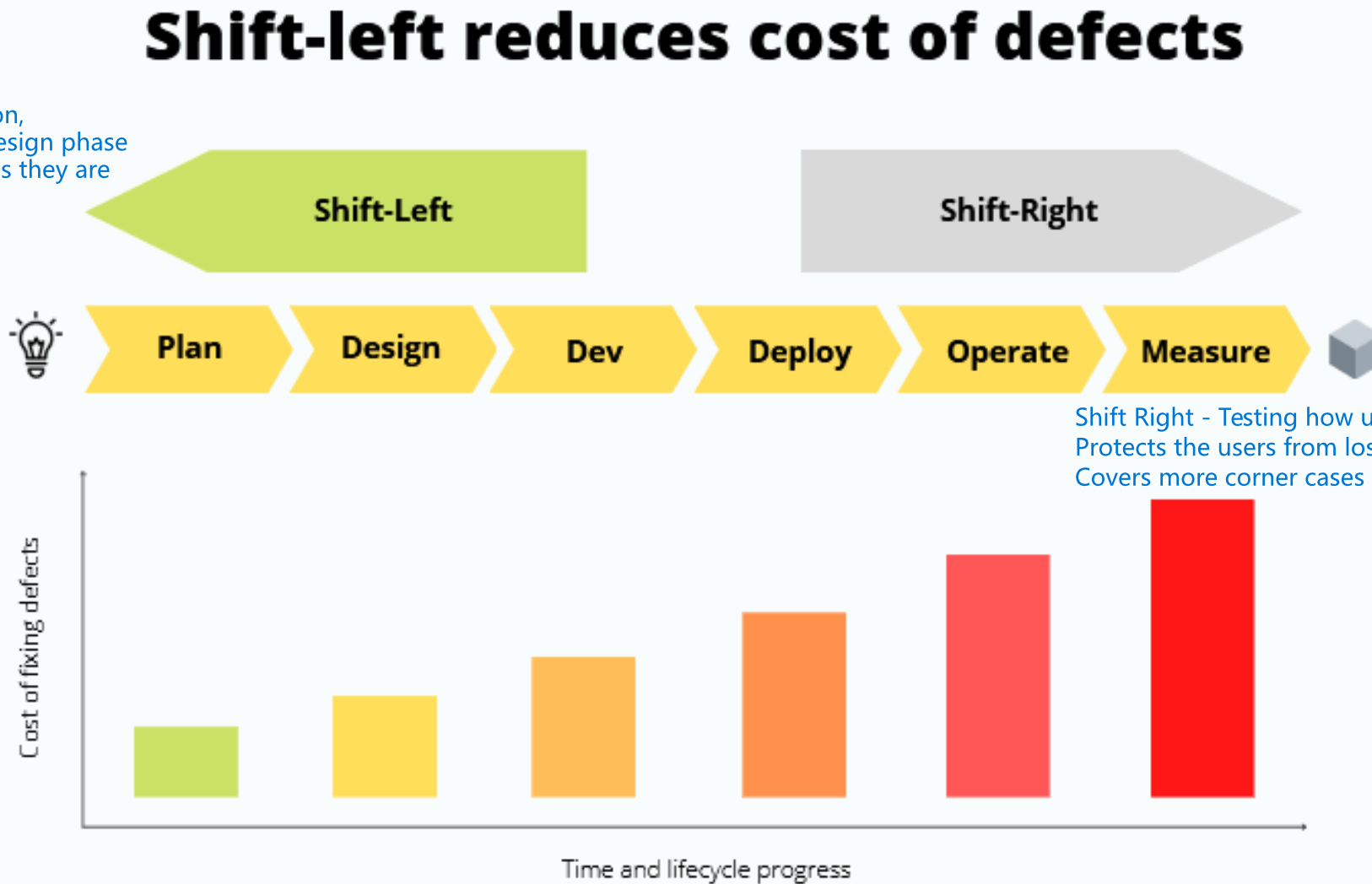
Exposed Feature Set + Rings



Rings are a set of features

Testing: Shift-Left & Shift-Right

Shift Left - Testing early on,
spending more time in design phase
Reduces cost of defects as they are
caught early on



Infrastructure Concepts

patterns & practices

proven practices for predictable results

This poster describes common problems in designing cloud applications, services and patterns that offer guidance on how to solve them. The information applies to Microsoft Azure as well as other cloud platforms. The scope of the top of each line represents the problem area that the pattern solves. Patterns that include code samples are indicated by the icon.

Visit <https://aka.ms/CloudDesignPatterns> to download.

Problem areas in the cloud

Availability

Availability defines the proportion of time that the system is functional and meeting its SLA. It is affected by system errors, hardware and software malfunctions, and system load. It is usually measured as a percentage of uptime. Cloud applications typically provide users with a service level agreement (SLA) which means that applications must be designed and implemented in a way that maintains availability.

For more info, see <https://aka.ms/CloudDesignPatterns/Availability>.

Data Management

Data management is the key element of cloud applications, and influences most of the quality attributes. Data is typically hosted in different locations and across multiple servers for reasons such as performance, availability or availability, and data is processed in a range of challenges. For example, data consistency must be maintained, and data will typically need to be synchronized across different locations.

For more info, see <https://aka.ms/CloudDesignPatterns/DataManagement>.

Design and Implementation

Cloud design encompasses factors such as consistency and coherence in component design and deployment, maintainability to simplify system evolution and development, and availability to ensure that the system is always available to be used in other applications and in other contexts. Decisions made during the design and implementation phase have a huge impact on the quality and the total cost of ownership of cloud-based applications and services.

For more info, see <https://aka.ms/CloudDesignPatterns/DesignAndImplementation>.

Messaging

The distributed nature of cloud applications requires a messaging infrastructure that supports the system's requirements. Messaging is a key element in many cloud applications, and is used to manage data, and provides many benefits, but also brings challenges such as the reliability of messages, message management, message delivery and more.

For more info, see <https://aka.ms/CloudDesignPatterns/Messaging>.

Management and Monitoring

Cloud applications run in a remote datacenter where you do not have full control of the infrastructure. In some cases, the operating system. This can make management and monitoring more difficult than on-premises deployment. Applications must expose metrics information that administrators and operators can use to manage and monitor the system, as well as supporting changing business requirements without requiring the application to be stopped or redeployed.

For more info, see <https://aka.ms/CloudDesignPatterns/ManagementAndMonitoring>.

Performance and Scalability

Performance is an indication of the responsiveness of a system, while scalability is the ability to gracefully handle increases in load, perhaps through an increase in the number of servers. Cloud applications are usually required to handle increases in load, and this can be challenging. Scalability is a key element in many cloud applications, and is used to manage data, and provides many benefits, but also brings challenges such as the reliability of messages, message management, message delivery and more.

For more info, see <https://aka.ms/CloudDesignPatterns/PerformanceAndScalability>.

Resiliency

Resiliency is the ability of a system to gracefully handle and recover from failures. The nature of cloud computing, where applications are often multi-tenant, use shared platform services, complete for resources and services, and are often exposed to the public, and may serve untrusted users. Applications must be designed and deployed in a way that protects them from malicious attacks, restricts access to only approved users, and protects sensitive data.

For more info, see <https://aka.ms/CloudDesignPatterns/Resiliency>.

Security

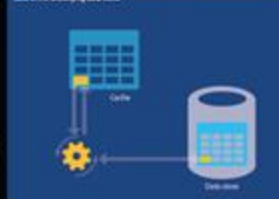
Security is the capability of a system to prevent malicious or accidental actions outside of the designed scope and to prevent disclosure of sensitive information. Cloud applications are exposed on the Internet outside trusted on-premises boundaries, are often open to the public, and may serve untrusted users. Applications must be designed and deployed in a way that protects them from malicious attacks, restricts access to only approved users, and protects sensitive data.

For more info, see <https://aka.ms/CloudDesignPatterns/Security>.

Cache-aside

Cache-aside

Load data on demand into a cache from a data store. This pattern can improve performance and also helps to maintain consistency between data held in the cache and the data in the underlying data store.



For more info, see <https://aka.ms/CloudDesignPatterns/CacheAside>.

Event Sourcing

Event Sourcing

Use an append-only store to record actions taken on data, rather than the current state and use the data to reconstruct the current state. This pattern can improve performance and also helps to maintain consistency between data held in the cache and the data in the underlying data store.



For more info, see <https://aka.ms/CloudDesignPatterns/EventSourcing>.

Leader Election

Leader Election

Coordinate the actions performed by a collection of collaborating peer instances in a distributed application by electing one instance as the leader that ensures regularity for managing the other instances. This pattern can help to ensure that the system is always available to be used in other applications and in other contexts.



For more info, see <https://aka.ms/CloudDesignPatterns/LeaderElection>.

Runtime Reconfiguration

Runtime Reconfiguration

Design an application that can be reconfigured without requiring redeployment of the application. This pattern can help to ensure that the system is always available to be used in other applications and in other contexts.



For more info, see <https://aka.ms/CloudDesignPatterns/RuntimeReconfiguration>.

Circuit Breaker

Circuit Breaker

Prevents faults that flow like a variable amount of time to notify when connecting to a service or resource. This pattern can improve the reliability and availability of an application.



For more info, see <https://aka.ms/CloudDesignPatterns/CircuitBreaker>.

External Configuration Store

External Configuration Store

Store configuration information out of the application deployment package in a centralized location. This pattern can help to ensure that the system is always available to be used in other applications and in other contexts.



For more info, see <https://aka.ms/CloudDesignPatterns/ExternalConfigurationStore>.

Materialized View

Materialized View

Store pre-computed data on the data in one or more data stores when the data is frequently accessed. This pattern can help to ensure that the system is always available to be used in other applications and in other contexts.



For more info, see <https://aka.ms/CloudDesignPatterns/MaterializedView>.

Scheduler Agent Supervisor

Scheduler Agent Supervisor

Coordinate a set of actions across a distributed set of services and other services using a scheduler agent. This pattern can help to ensure that the system is always available to be used in other applications and in other contexts.



For more info, see <https://aka.ms/CloudDesignPatterns/SchedulerAgentSupervisor>.

Compensating Transaction

Compensating Transaction

Use the work performed by a series of steps, which together define an eventually consistent transaction. If one or more of the steps fails, the transaction is rolled back and the work performed by the steps is compensated by a series of steps.



For more info, see <https://aka.ms/CloudDesignPatterns/CompensatingTransaction>.

Federated Identity

Federated Identity

Delegate authentication to an external identity provider. This pattern can help to ensure that the system is always available to be used in other applications and in other contexts.



For more info, see <https://aka.ms/CloudDesignPatterns/FederatedIdentity>.

Pipes and Filters

Pipes and Filters

Decompose a task that performs complex processing into a series of discrete elements that can be reused. This pattern can help to ensure that the system is always available to be used in other applications and in other contexts.



For more info, see <https://aka.ms/CloudDesignPatterns/PipesAndFilters>.

Sharding

Sharding

Divide a data store into a set of horizontal partitions or shards. This pattern can help to ensure that the system is always available to be used in other applications and in other contexts.

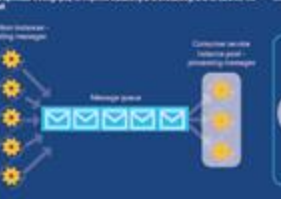


For more info, see <https://aka.ms/CloudDesignPatterns/Sharding>.

Competing Consumers

Competing Consumers

Builds multiple independent consumers to process messages received in the same message channel. This pattern can help to ensure that the system is always available to be used in other applications and in other contexts.



For more info, see <https://aka.ms/CloudDesignPatterns/CompetingConsumers>.

Gatekeeper

Gatekeeper

Protect applications and services by using a dedicated host instance that acts as a broker between clients and the application or service, and handles requests and responses.



For more info, see <https://aka.ms/CloudDesignPatterns/Gatekeeper>.

Priority Queue

Priority Queue

Handle requests sent to a service in a buffer between a task and a service that it handles in order of their priority. This pattern can help to ensure that the system is always available to be used in other applications and in other contexts.



For more info, see <https://aka.ms/CloudDesignPatterns/PriorityQueue>.

Static Content Hosting

Static Content Hosting

Deploy static content to a cloud-based storage service that can deliver them directly to end users. This pattern can help to ensure that the system is always available to be used in other applications and in other contexts.

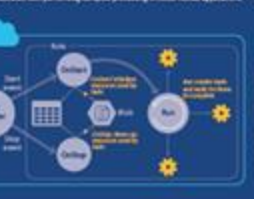


For more info, see <https://aka.ms/CloudDesignPatterns/StaticContentHosting>.

Compute Resource Consolidation

Compute Resource Consolidation

Consolidate multiple tasks or operations into a single computational job. This pattern can help to ensure that the system is always available to be used in other applications and in other contexts.



For more info, see <https://aka.ms/CloudDesignPatterns/ComputeResourceConsolidation>.

Health Endpoint Monitoring

Health Endpoint Monitoring

Implement a health check within an application that external tools can access through a dedicated endpoint or service. This pattern can help to ensure that the system is always available to be used in other applications and in other contexts.



For more info, see <https://aka.ms/CloudDesignPatterns/HealthEndpointMonitoring>.

Queue-Based Load Leveling

Queue-Based Load Leveling

Use a queue to act as a buffer between a task and a service that it handles in order to smooth intermittent heavy loads that may often also cause the service to fail on the task. This pattern can help to ensure that the system is always available to be used in other applications and in other contexts.

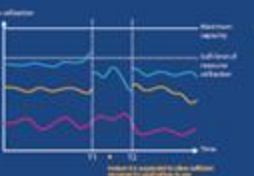


For more info, see <https://aka.ms/CloudDesignPatterns/QueueBasedLoadLeveling>.

Throttling

Throttling

Control the consumption of resources used by an instance of an application, or individual components of the application, to ensure that the system is always available to be used in other applications and in other contexts.



For more info, see <https://aka.ms/CloudDesignPatterns/Throttling>.

Command and Query Responsibility Segregation (CQRS)

Command and Query Responsibility Segregation (CQRS)

Separate operations that read data from operations that update data by using separate models. This pattern can help to ensure that the system is always available to be used in other applications and in other contexts.



For more info, see <https://aka.ms/CloudDesignPatterns/CQRS>.

Index Table

Index Table

Create a table on the data in a data store that is frequently referenced by query patterns. This pattern can help to ensure that the system is always available to be used in other applications and in other contexts.



For more info, see <https://aka.ms/CloudDesignPatterns/IndexTable>.

Retry

Retry

Enable an application to handle anticipated temporary failure when it attempts to interact with an external service by temporarily storing the operation that has failed. This pattern can help to ensure that the system is always available to be used in other applications and in other contexts.



For more info, see <https://aka.ms/CloudDesignPatterns/Retry>.

Valet Key

Valet Key

Use a token or key that provides access with restricted access to a specific resource. This pattern can help to ensure that the system is always available to be used in other applications and in other contexts.



For more info, see <https://aka.ms/CloudDesignPatterns/ValetKey>.

Cloud Design Patterns: Prescriptive Architecture Guidance for Cloud Applications



Like it? Get it.

<https://aka.ms/CloudDesignPatterns>

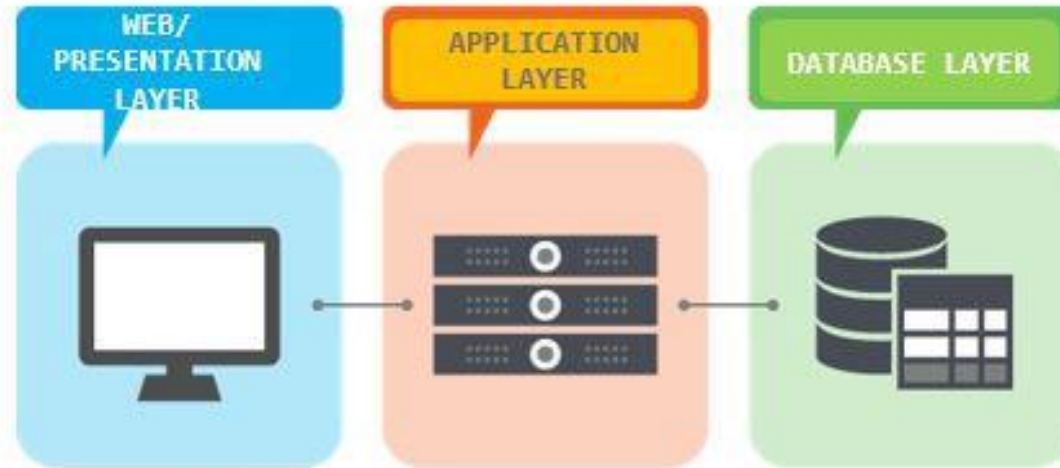


© 2015 Microsoft Corporation. All rights reserved.

pattern@microsoft.com

Basic 3-tier layered architecture

 *Often sufficient for many applications!*



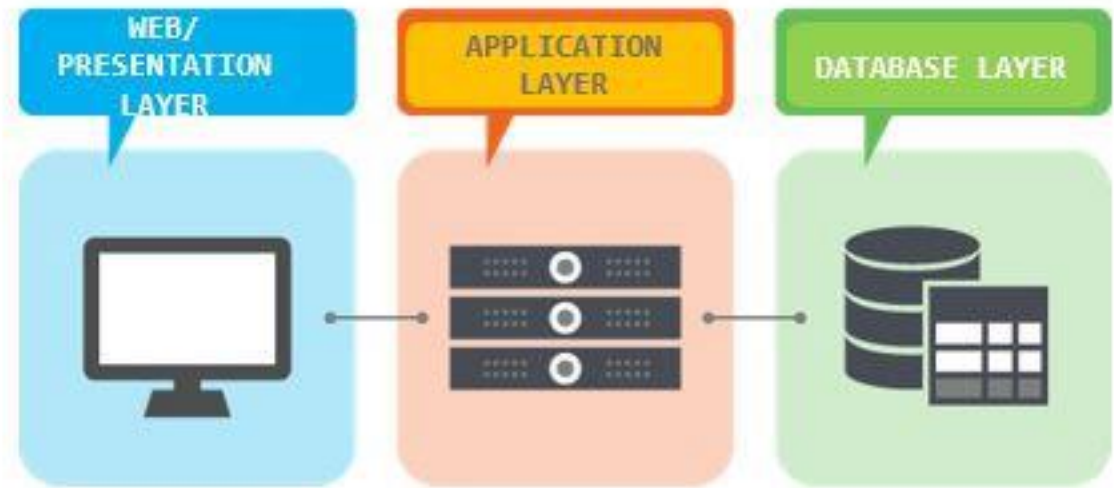
User opens login page, to withdraw money from account.

 *Page: Makes REST request to application server, renders result on screen.*

 *App server performs authentication and retrieves account data, coordinates transaction.*

 *Database: Retrieves data and processes*

When not enough?



 All users opens login page, to withdraw money from account.

 App server is unplugged or VM is deleted.



 Disks are full and incoming data requests exceeds capacity.



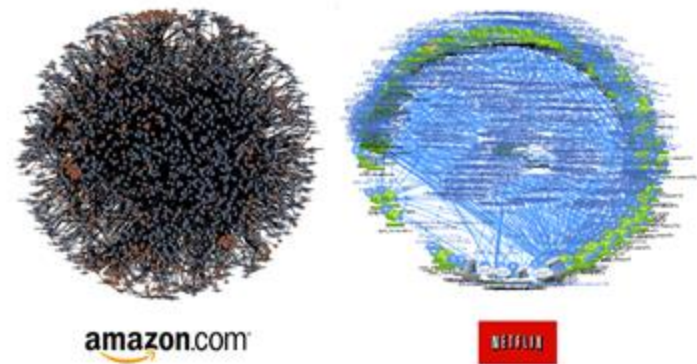
Towards Architecture 2.0

Traditional architectural styles are insufficient for modern web-scale workloads.

 *Moving forward, we have two options.*



Patch architectures



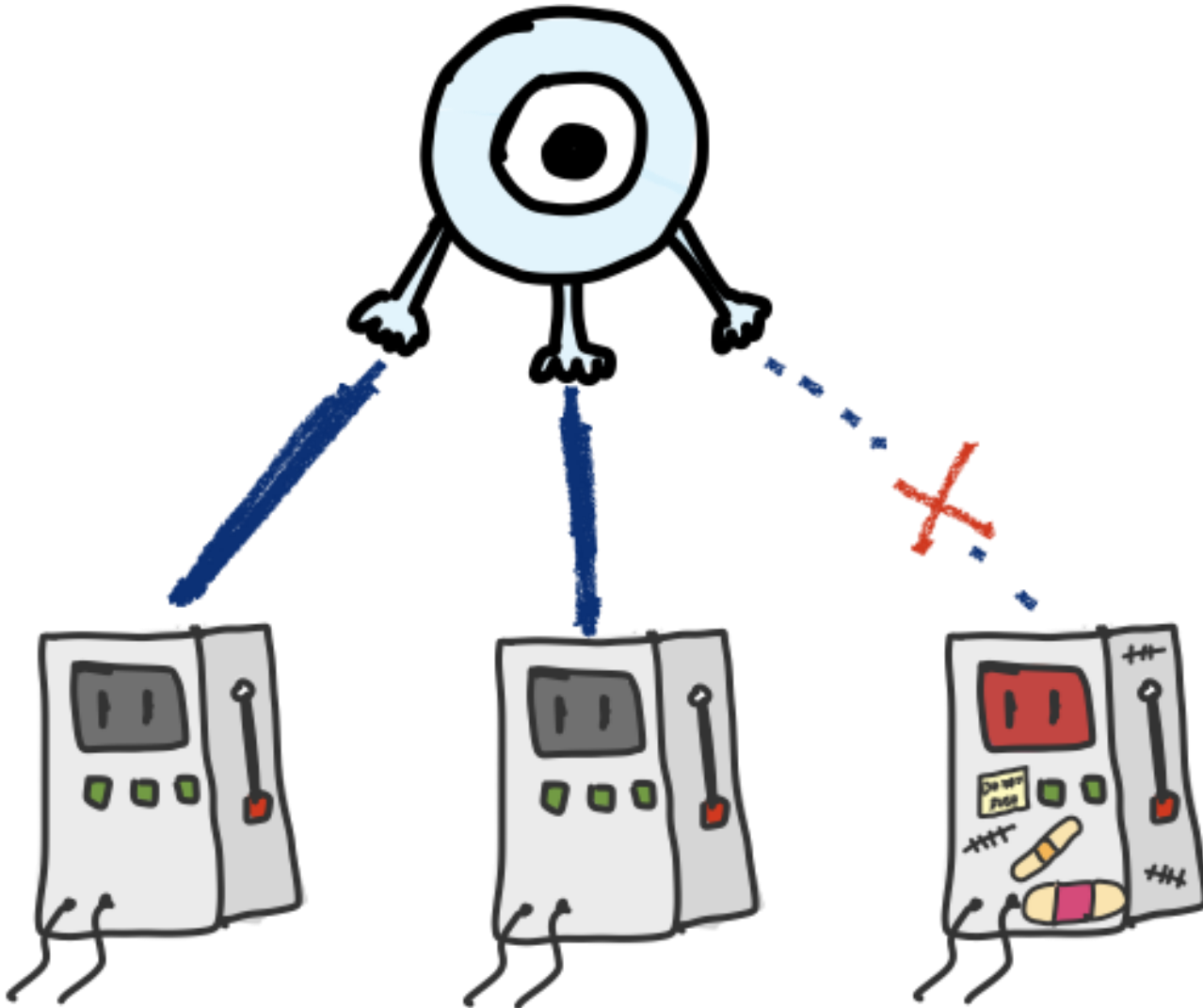
Novel architectures

Desirable Properties of Modern Infrastructure

1. Available: No or limited interruption to provided services.
2. Scalable: Can increase specific units in response to demand.
3. Efficient: Avoid redundant work, shift responsibility to low-cost components.
4. Robust: Graceful response to failure of infrastructure or services.
5. Durable: Operations and data persistent across partitions.
6. Regulation: Can throttle incoming work into component.
7. Isolation: Can reduce impact of component behavior on others.
8. Resilient: Can adapt to unknown and unexpected situations (*what happens region goes down, ... or if Google kills GCP?*)

Common infrastructure components

Load Balancers.

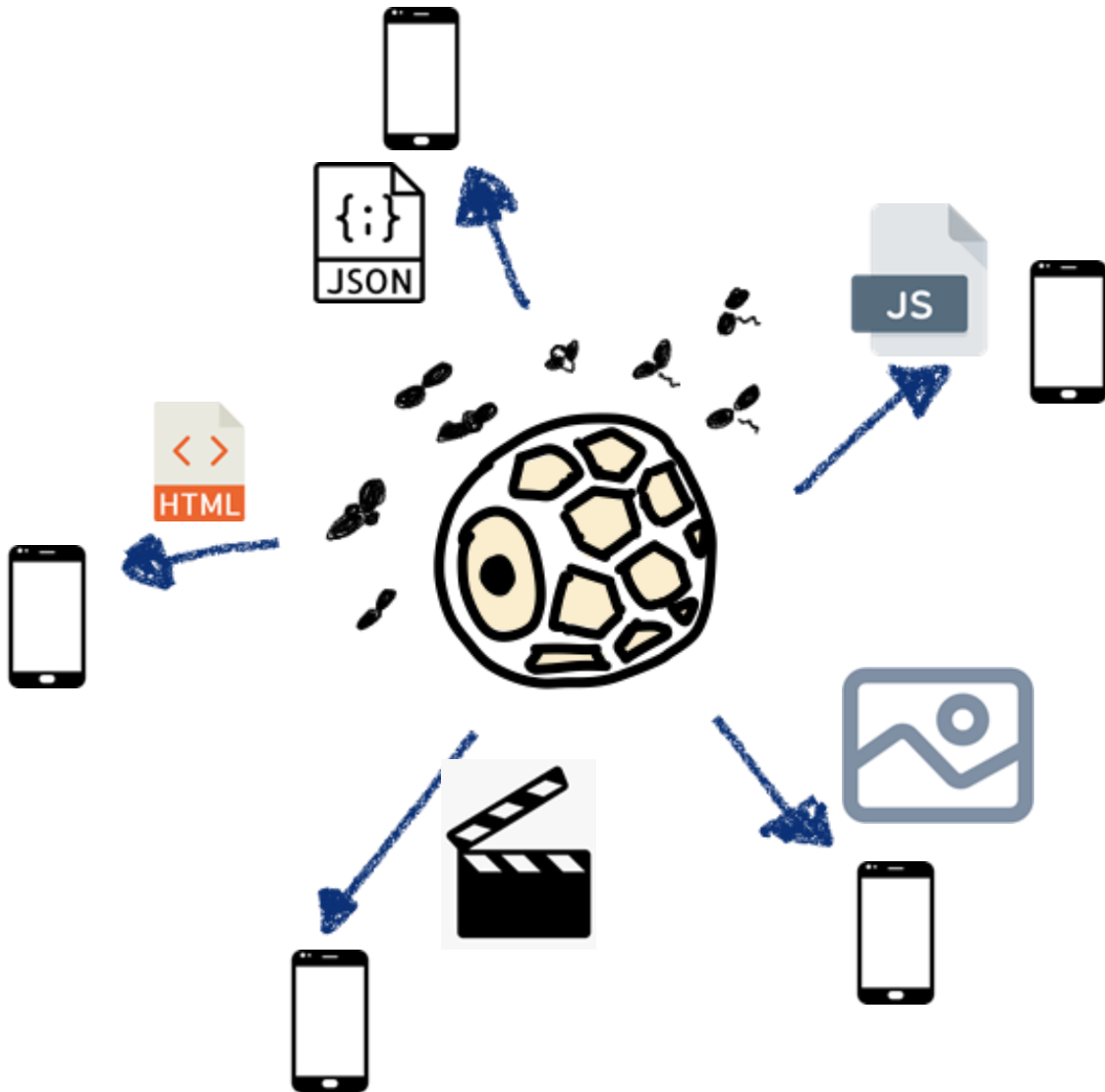


Send traffic to healthy instances.
Request new instances as needed.

Ensures availability and scalability.



Content Delivery Network and Caches

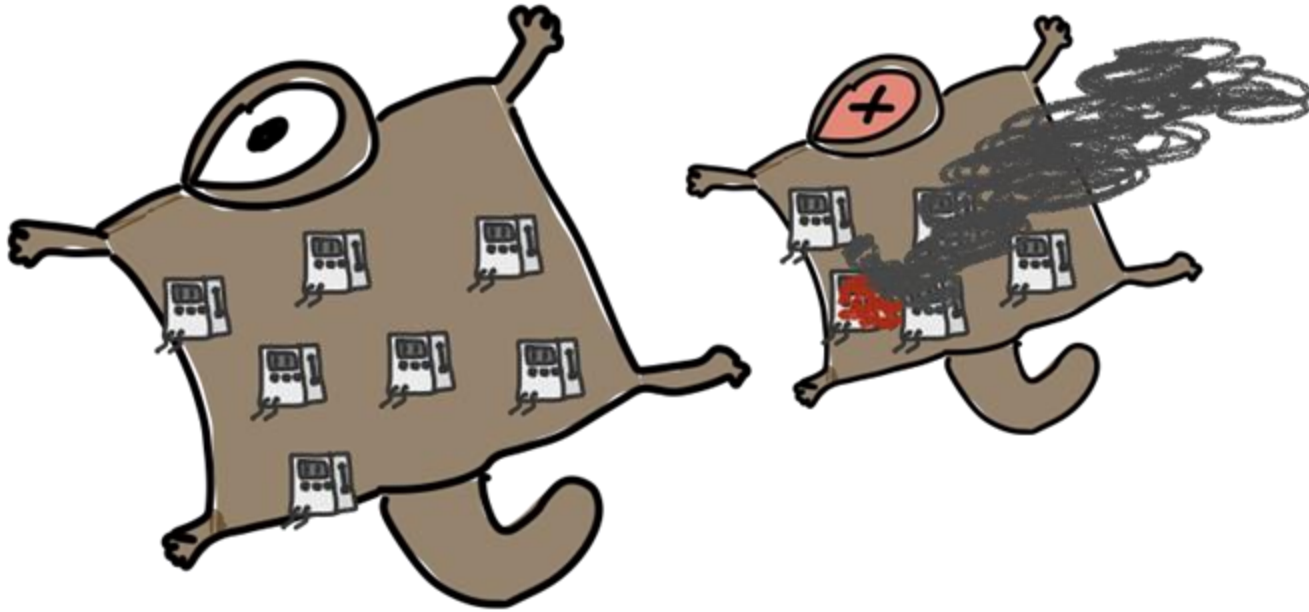


Shift delivery of *static content* from infrastructure to third-party provider.

Ensures availability and



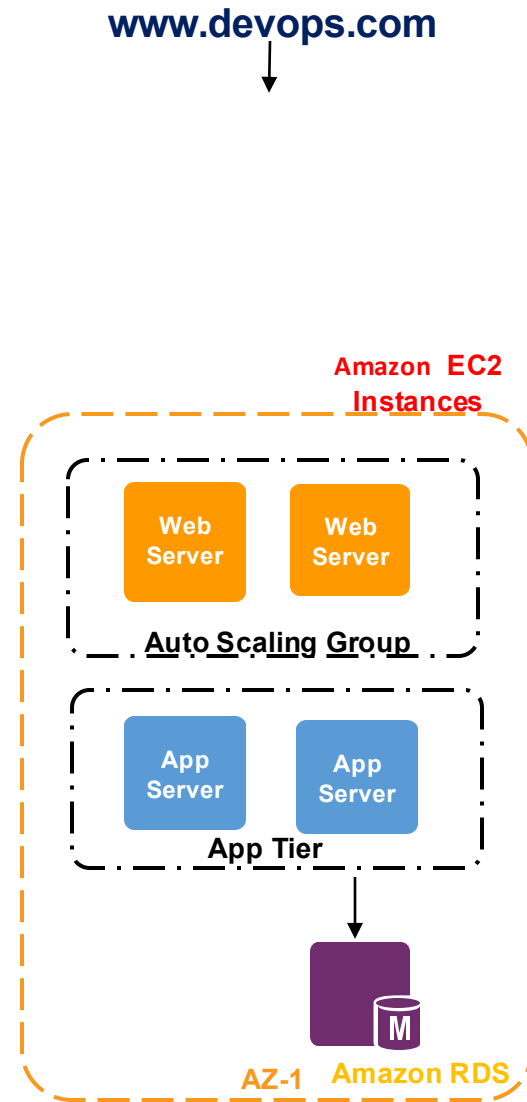
Zones



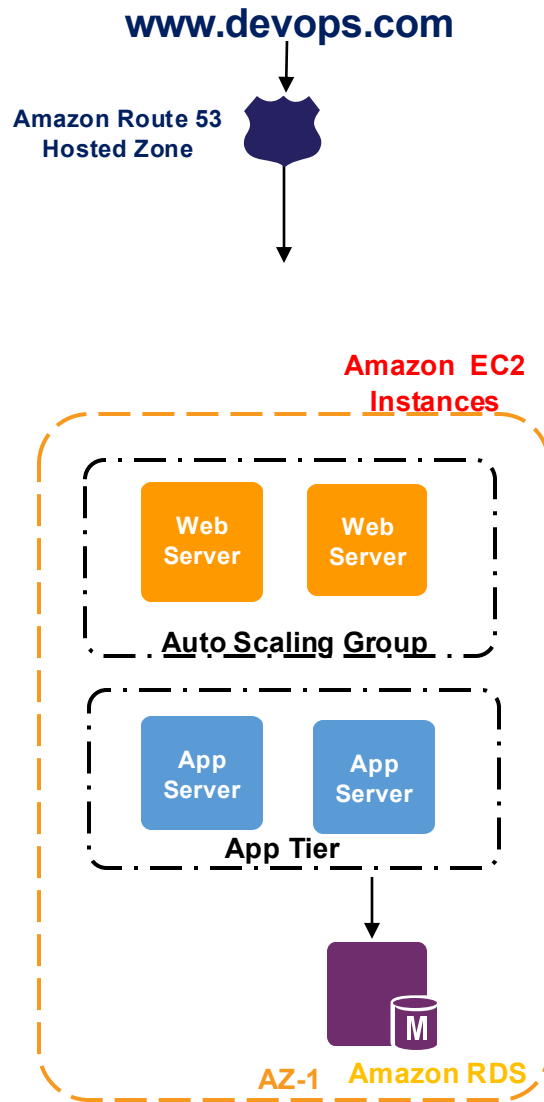
Supports isolation and spreads risk by operating instances in independent pools.

Ensures availability and robustness. Useful for supporting certain deployment strategies.

Patching our 3-tier architecture

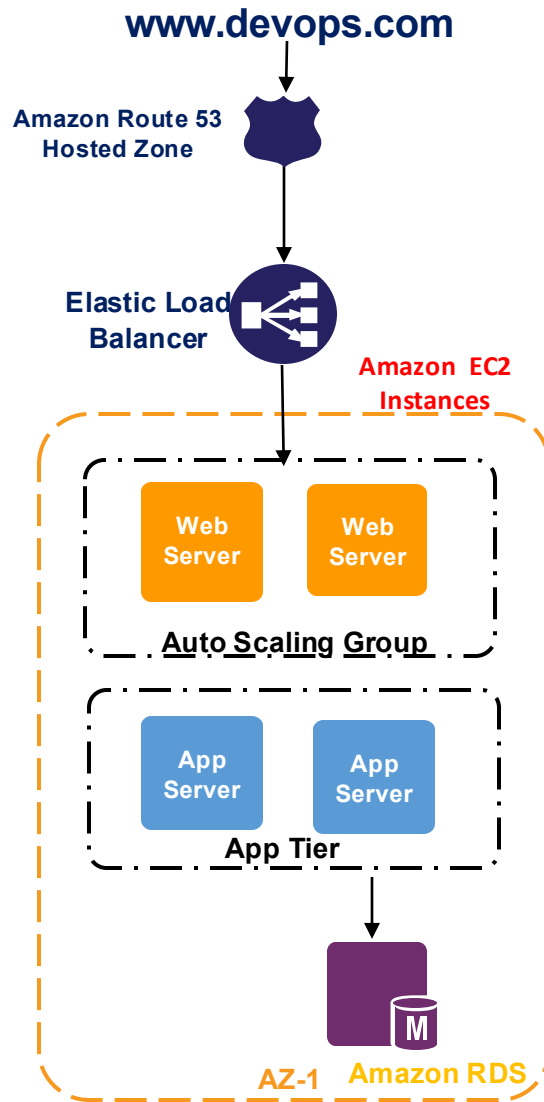
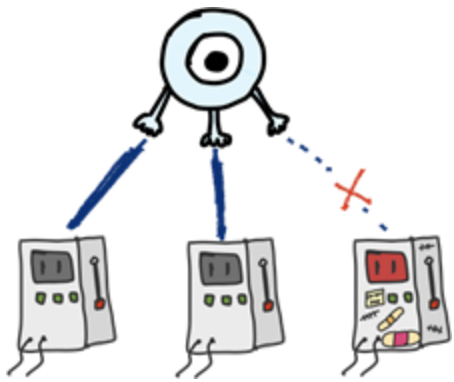


Making Available and Scalable



DNS Services

- *Simple routing service.*
- Turn a request to www.devops.com into an ip address, projected into AWS infrastructure.
- Routing Options:
 - Latency Based Routing
 - Geo DNS
 - Weighted Round Robin
- ***Simple round robin:***
www0.devops.com,
www1.devops.com, ...



Load Balancing

- *Dynamic balancing and scaling*
- Ensure only healthy instances receive requests.
- Not only can route request, can request new auto-scaling instance if needed.
- *Condition:*
 - *If latency greater than 4 seconds,*
 - *if CPU utilization greater than 70%*
 - *Request count, 5xx response count*

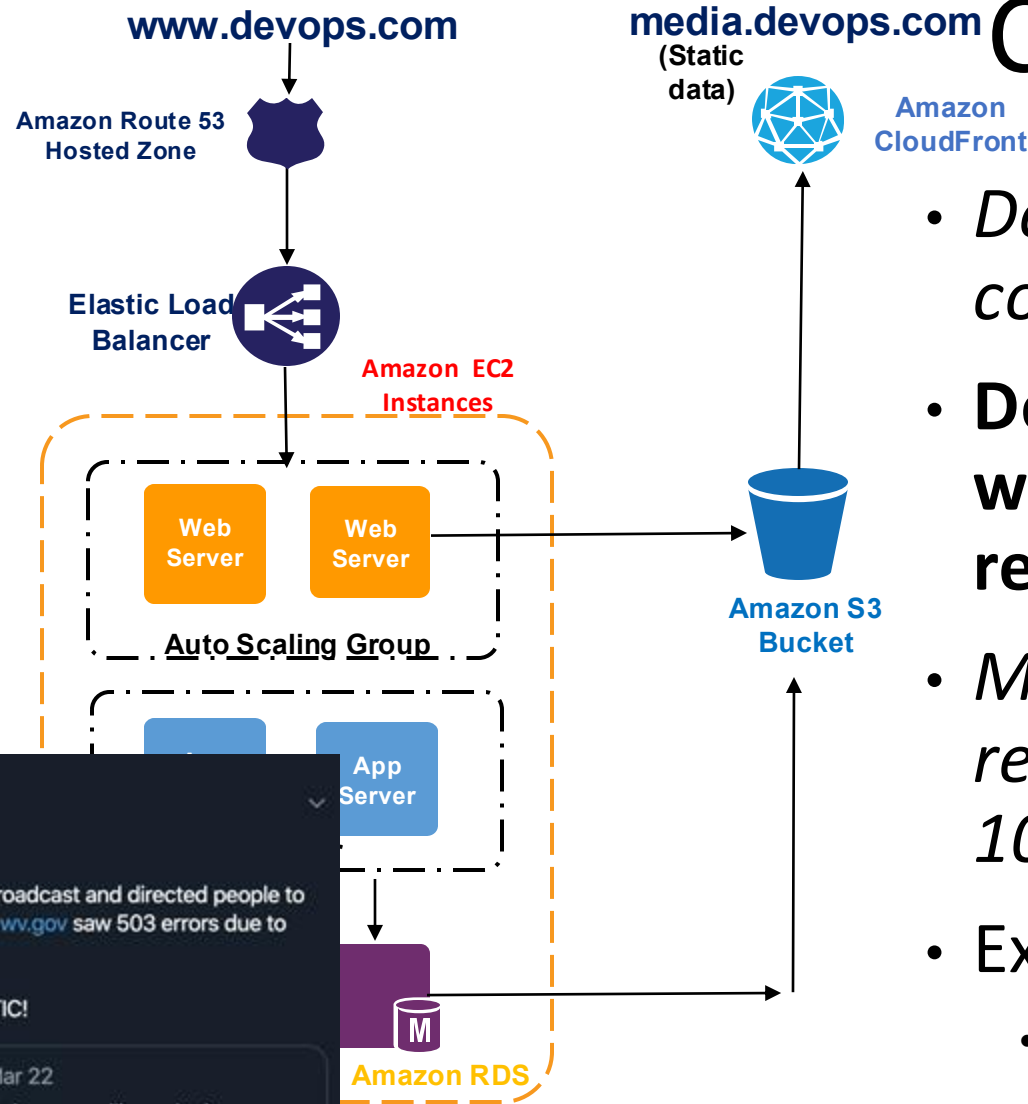
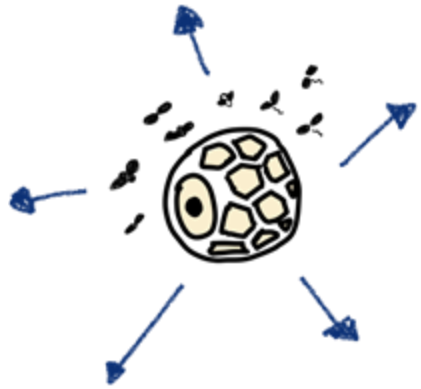
What happens when load balancer goes down?



Leader election: Stand-by servers can take over in the event leader goes down.



Ensuring efficiency and regulation



Content Delivery

- *Deliver frequently accessed content*
- **Don't waste CPU, sockets, and worker threads on static requests.**
- *Most common DDOS attack is to request a large static file, e.g., a 100 MB PDF file.*
- Example content:
 - Javascript libraries/CSS
 - Images, videos
 - Popular objects

SignpostMarv Retweeted



Adam Johnson @adamj_design · 31m
Yesterday, I saw this tweet and chuckled.

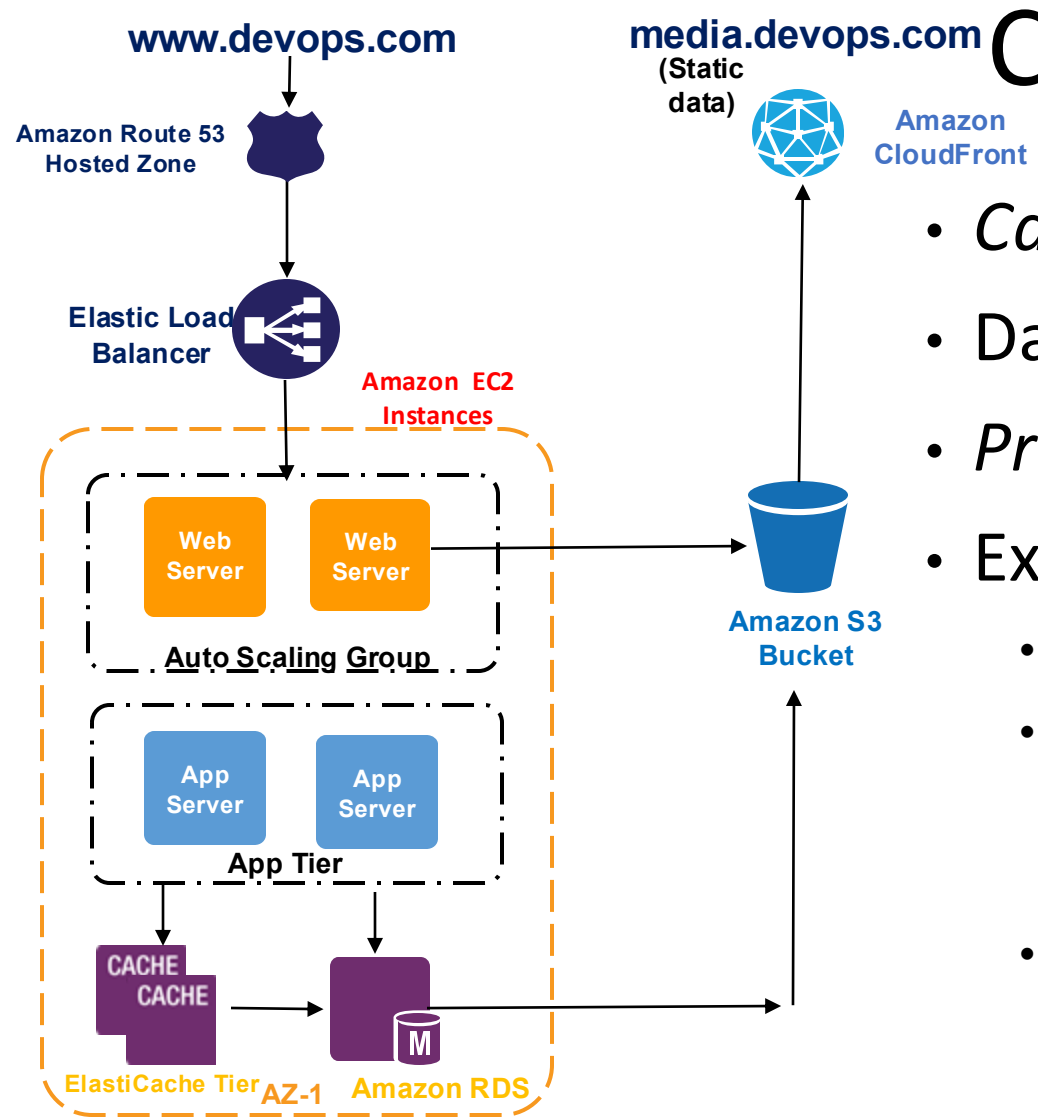
An hour later, the WV Governor televised a broadcast and directed people to wv.gov for coronavirus info. Almost instantly, wv.gov saw 503 errors due to load.

👉 No more chuckling for me. MAKE IT STATIC!

Eric Meyer, CSS Nerd @meyerweb · Mar 22

If you are in charge of a health-related web site, or a utility web site, or any site that has information people need right now: MAKE IT STATIC. As much as you can, convert pages to static HTML/CSS. Yes, even that part.

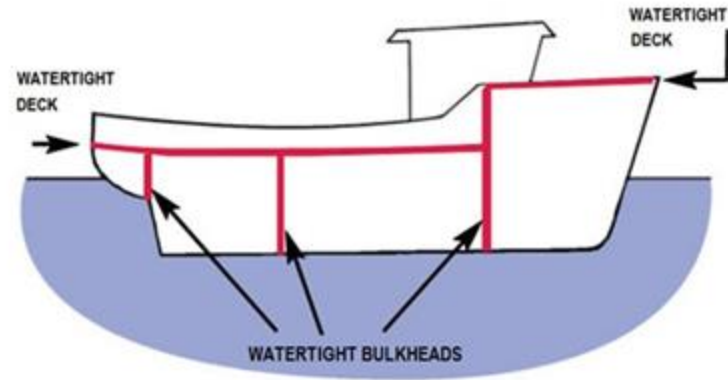
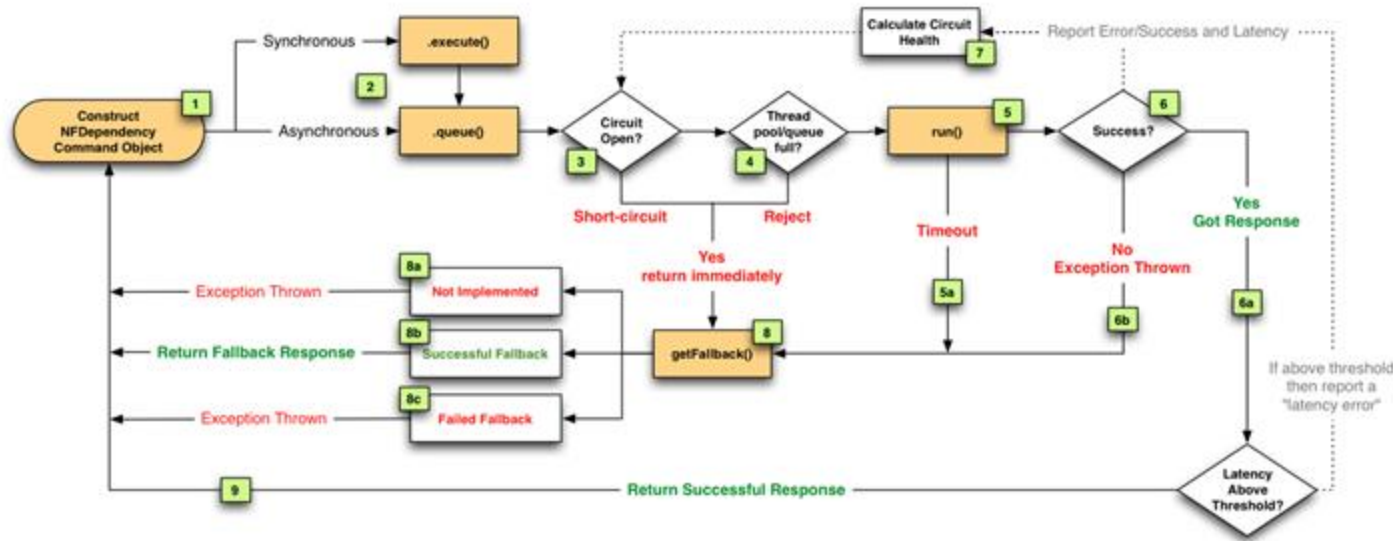
[Show this thread](#)



Caches and Queues

- *Cache popular dynamic requests*
- Data is **in-memory, not on disk.**
- *Providers: Memcached, Redis.*
- Example content:
 - Front page of reddit
 - Computationally intensive requests: *Driving directions from Raleigh to Florida (during Spring Break).*
 - *Queues: Data waiting to be processed and stored by back-end.*

Circuit breakers and bulkheads



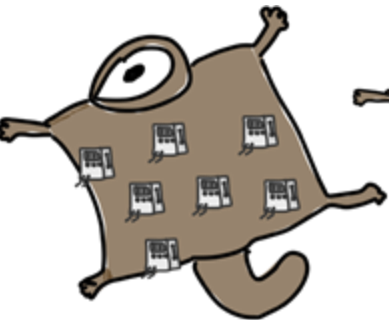
Bulkheads effectively isolate components and protect from cascading failures through the enforcement of limits (**load shedding**).

<https://martinfowler.com/bliki/CircuitBreaker.html>

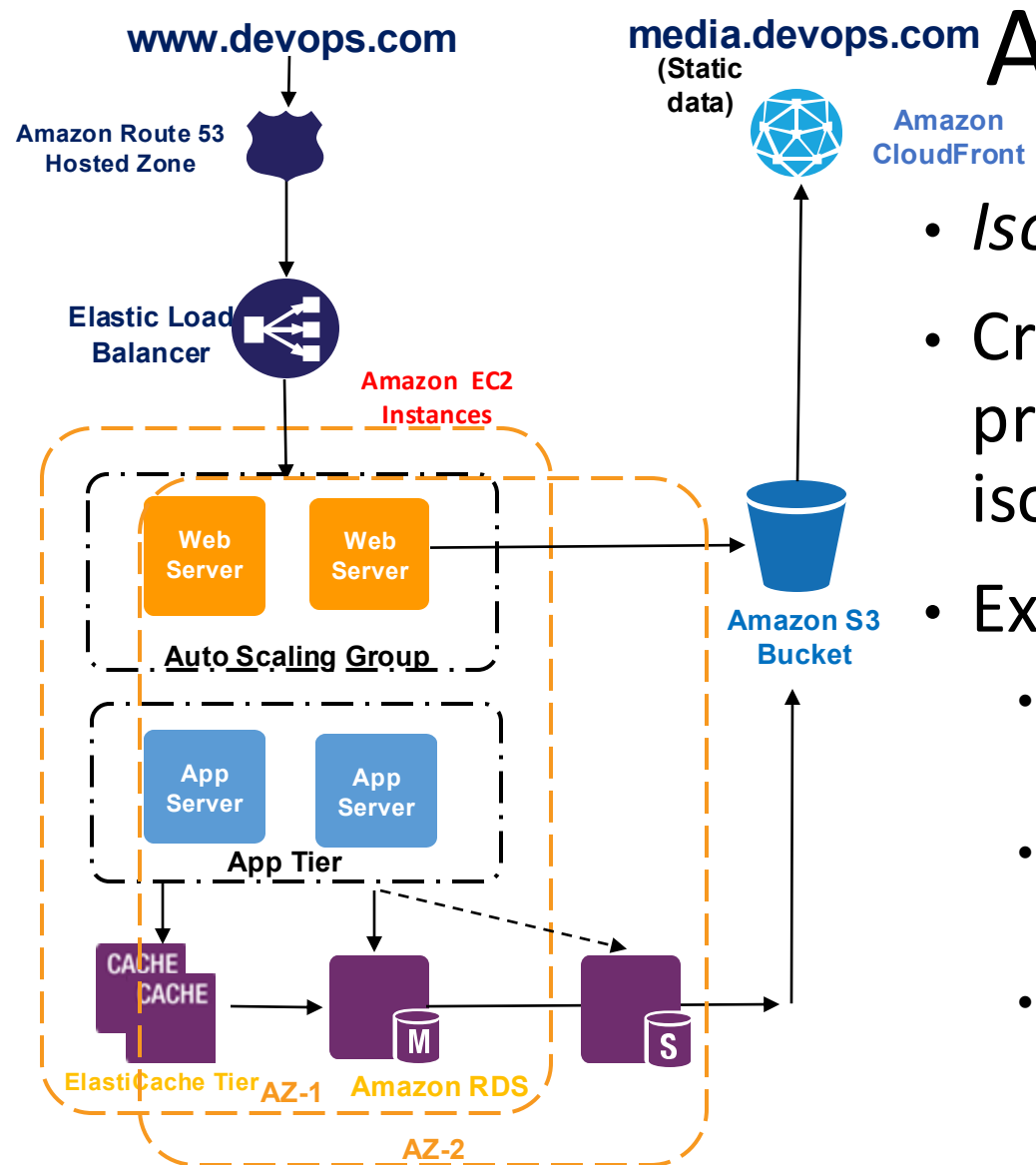
<https://medium.com/dm03514-tech-blog/sre-resiliency-bulkheads-in-action-using-js-453c02a940c0>

Example: No matter how many requests are applied NGINX has a hard limit on the number of processes it will create

Ensuring Isolation

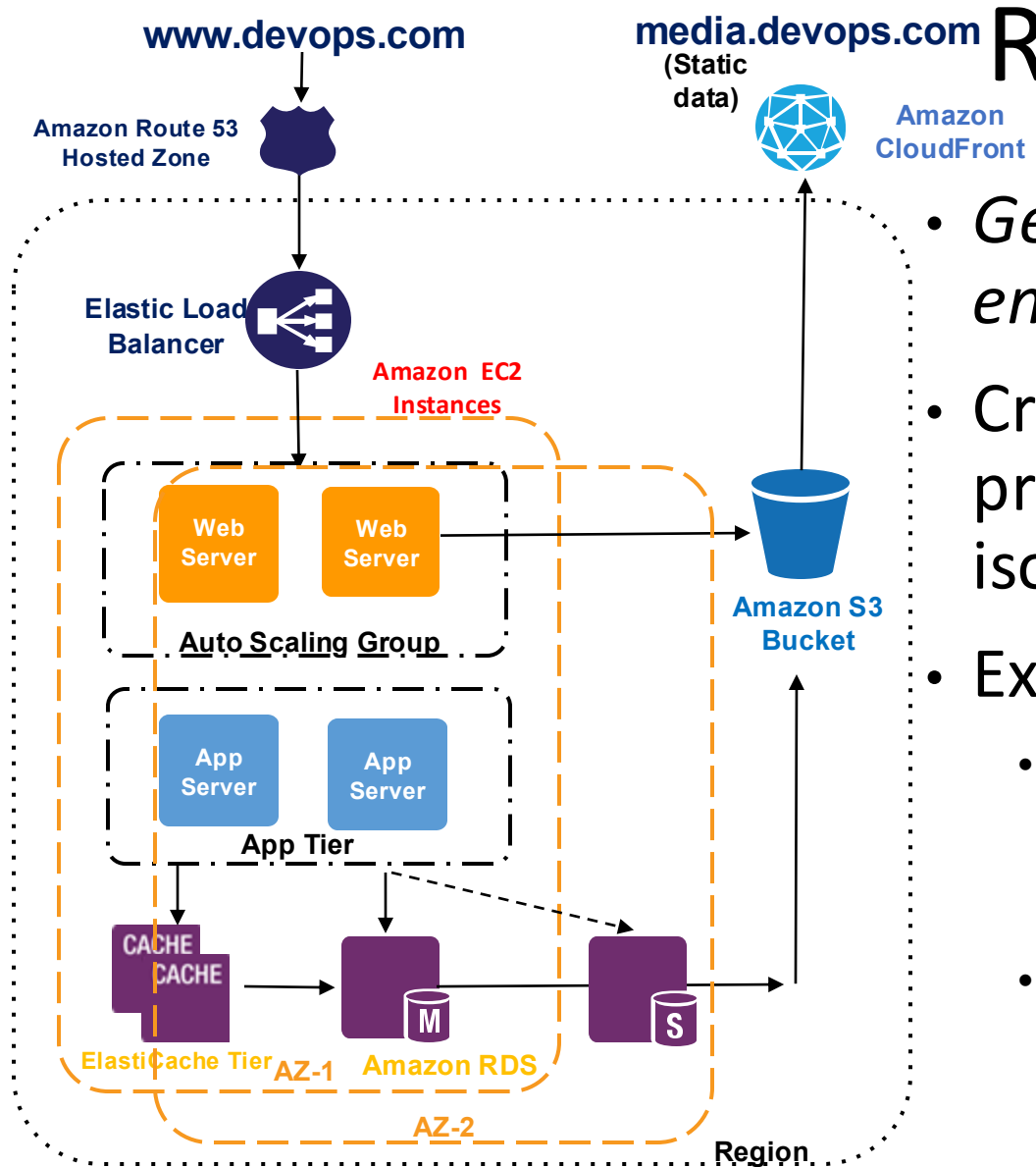


Cascading failures



Availability Zones

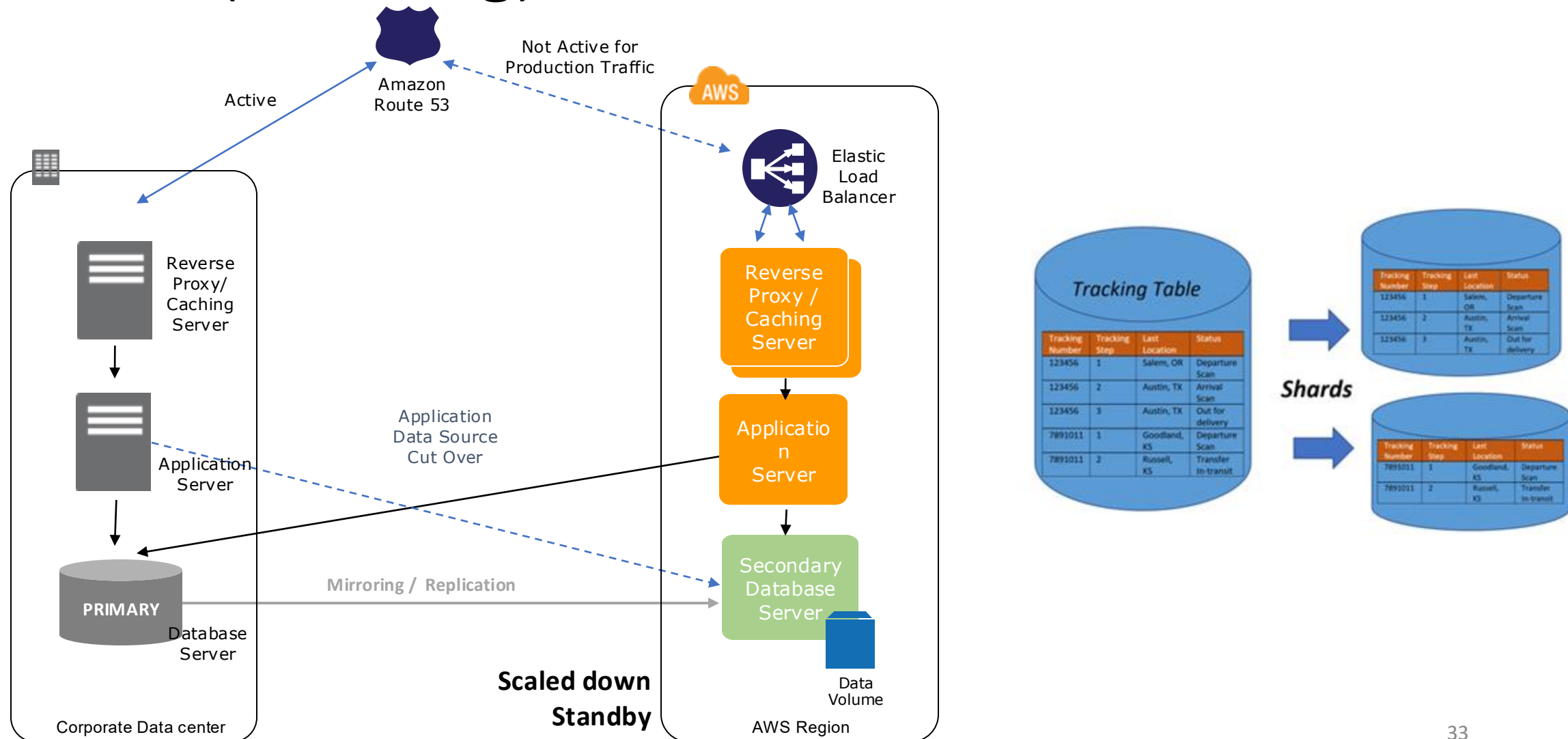
- *Isolated production environment*
- Create pools or zones of your production environment that are isolated from other instances.
- Example use cases:
 - Switch requests when deploying new version,
 - Have environments resilient to pollution (corrupted cache),
 - Avoid slow spin up: Anticipate large traffic boost (lunch/primetime hours)



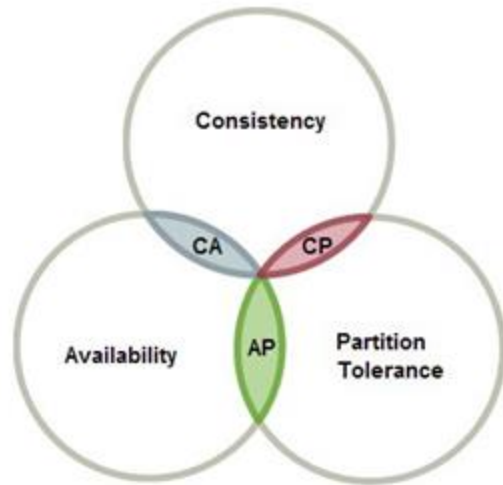
Regions

- *Geographically placed production environments*
- Create pools or zones of your production environment that are isolated from other instances.
- Example use cases:
 - Serve traffic closest to where it occurs (Asia, Europe, South America, etc.)
 - Handle different laws, content distribution rules

Ensuring durability (mirroring) and scalability and isolation (sharding)



Limitations: CAP theorem

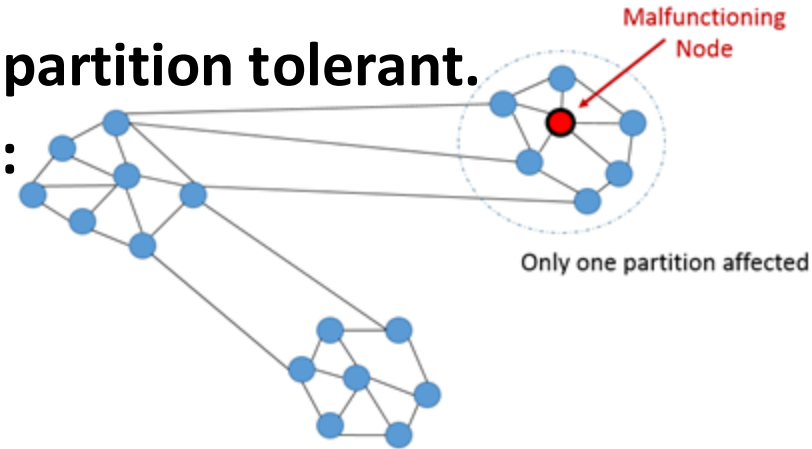


A distributed database system can only have 2 of the 3 properties:

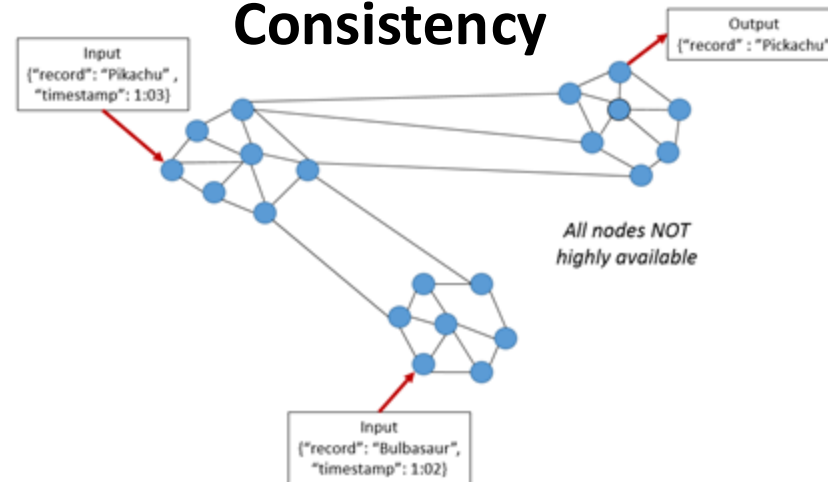
Consistency, Availability and Partition Tolerance

Assume partition tolerant.

Pick one:



Consistency



Availability



Microservices

Split services into single deployable units.

Monolith

hello/
users/
appointments/
...

Run single application with

Microservices

1x hell
o/
30x users
/
10x appointment
s/

Run each service in its own

```
import org.springframework.boot.*;
import org.springframework.boot.autoconfigure.*;
import org.springframework.stereotype.*;
import org.springframework.web.bind.annotation.*;

@RestController
@EnableAutoConfiguration

public class Example {

    @RequestMapping("/api/hello")
    String hello() { return "Hello World!"; }

    public static void main(String[] args) throws
        Exception {

        SpringApplication.run(Example.class,
            args);
    }
}
```

Scalability: Can run as many instances of particular services as needed across multiple servers.

Robustness and Isolation: Can still run other services if one fails.

Microservices Architecture (Netflix)



All Netflix Content in One Box

Open Connect

<http://gizmodo.com/this-https://openconnect.netflix.com/en/box-can-hold-an-entire-netflix-1502500450>

CDN:
Open connect for video data.

Regions:
us-east-1
us-west-2
eu-west-1

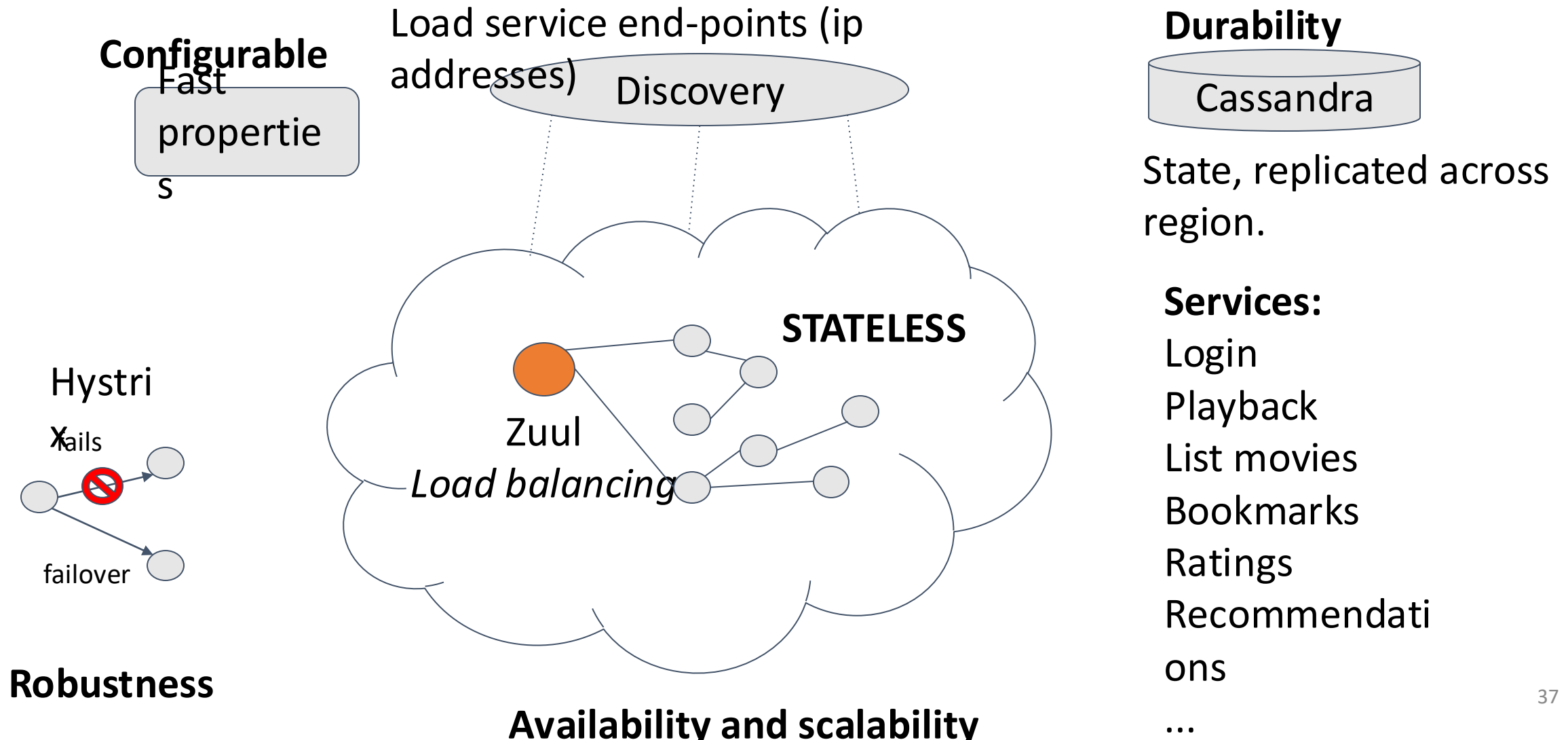
Routing:
dns/ip geo

AWS doesn't handle *any video data*.

AWS does discovery, load balancing, recommendations, profiles, 1 billion metrics streaming to Atlas, etc.

Video is all co-located with ISPs.

Microservices Architecture (Netflix)



Platform and Vendor Risks

- <https://blog.heroku.com/archives/2015/5/7/heroku-free-dynos>

"Heroku kills free hosting tier, replacing it with a new service that can only run 18 hours per day"

- Your Linodes have been assigned a maintenance window in which a reboot will occur. These times are listed within the Linode Manager[1] in the timezone set in your user's My Profile[2]. Your schedule in UTC timezone is as follows...

Unfortunately, due the logistical demands of this effort, your assigned windows are not changeable and the host reboots are mandatory.



In Summary

- Deployment Rings / Blast Radius
- Testing Shift Left / Shift Right
- Intro to Infrastructure Concepts
- Review of the 3-Layered Architecture and why it's not enough
- Desirable properties of Modern Web Architecture
- Availability Zones / Regions / Mirroring Sharding
- Circuit Breakers / Content Delivery Networks